
Teaching Speculative Decoding: Making Autoregressive Decoding Faster While Preserving Token Quality

Ayush Vikram

Paul G. Allen School of Computer Science & Engineering
University of Washington
avikram2@uw.edu

Grace Gong

Computer Science
Georgia Institute of Technology
ggong30@gatech.edu

1. The concept. Conventional autoregressive decoding is bottlenecked: generating each output token requires reloading the full model’s weights, and because every output token depends on the ones before it, decoding cannot be parallelized across the sequence. This makes decode memory-bandwidth bound, leaving most of the GPU’s compute capacity idle. Speculative decoding is a *lossless* algorithm that closes this gap by accepting multiple output tokens per expensive target-model evaluation. A smaller, cheaper *draft model* proposes several tokens ahead; the large *target model* verifies all of them in a single parallel forward pass, at roughly the wall-clock cost of generating just one token, since verification is exactly the same memory-bound operation whether it scores one candidate or several. Each proposed token is then accepted or rejected via rejection sampling: accept with probability $\min(1, q(x)/p(x))$ where p is the draft’s probability and q the target’s; upon the first rejection, a replacement token is drawn from the residual distribution $\text{norm}(\max(0, q - p))$, and every later draft prediction is discarded because it was conditioned on the rejected token. This accept/reject-with-residual-resampling rule, derived from the classical rejection-sampling identity, guarantees the resulting output distribution is *exactly identical* to sampling directly from the target model: nothing is approximated, and the only cost of a wrong guess is wasted draft compute. The single number that governs how much speedup this yields in practice is the acceptance rate α , the probability that a given draft token survives verification. A well-aligned draft model, one whose distribution p tracks the target’s distribution q closely, produces a high α and lets many tokens through per round; a poorly-aligned draft model produces a low α , so rejections happen early and often and the achievable speedup shrinks even though correctness is never at risk. This is why the technique has been adopted directly into production LLM-serving systems such as vLLM and TensorRT-LLM, and why frontier variants (EAGLE, EAGLE-2, Medusa) now compete primarily on raising α through better-aligned draft signals rather than by changing the correctness guarantee itself.

2. Leveling and prerequisite knowledge. Designed for **advanced undergraduate or graduate** students who have completed an introductory course covering autoregressive language models and basic probability (conditional distributions, sampling, and expectation). No prior systems or hardware background is required; the memory-bandwidth argument is developed from first principles in the lecture slides. The material is suitable as a single lesson within a larger LLM-systems, NLP, or generative-modeling course, or as a standalone module in an ML-systems reading group.

3. Learning objectives and outcomes. By the end of this lesson, a learner will be able to: (*remember*) state the two-model draft/target structure and the role of the acceptance ratio $q(x)/p(x)$; (*understand*) explain why parallel verification is cheap relative to sequential decoding, and why residual resampling (not simply re-sampling from q alone) is required to preserve the target model’s exact output distribution; (*apply*) implement the accept/reject and residual-resampling steps correctly from a written specification, matching a provided reference on synthetic distributions; (*analyze*) decompose an observed speedup into a draft-quality axis (α , the per-token acceptance rate) and a speculation-length axis (k), and explain why expected accepted tokens per round, $(1 - \alpha^{k+1})/(1 - \alpha)$, plateaus as k grows; (*evaluate*) given a fixed compute budget, argue whether investing in draft-model

quality (raising α) or in speculation depth (raising k) yields the better return, and justify the choice; (*create*) design a tree-structured extension that verifies multiple candidate continuations per target-model evaluation, in the style of EAGLE-2’s dynamic draft trees or Medusa’s multiple decoding heads, and reason about the additional bookkeeping a tree-attention mask requires over the linear case.

4. Linked papers (2022–2026, active frontier use).

- Y. Leviathan, M. Kalman, Y. Matias. *Fast Inference from Transformers via Speculative Decoding*. ICML 2023. (Origin of the algorithm taught here.)
- C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, J. Jumper. *Accelerating Large Language Model Decoding with Speculative Sampling*. arXiv:2302.01318, 2023. (Independently-derived, concurrent formulation.)
- Y. Li, F. Wei, C. Zhang, H. Zhang. *EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty*. ICML 2024. (Feature-reuse drafting.)
- Y. Li, F. Wei, C. Zhang, H. Zhang. *EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees*. EMNLP 2024. (Dynamic tree-structured speculation; the direct model for the tree-based Create objective above.)
- T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, T. Dao. *Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads*. ICML 2024. (Self-speculation via extra prediction heads; no separate draft model.)
- R. Svirschevski, A. May, Z. Chen, B. Chen, Z. Jia, M. Ryabinin. *SpecExec: Massively Parallel Speculative Decoding for Interactive LLM Inference on Consumer Devices*. NeurIPS 2024. (Wide speculation trees for RAM/disk-offloaded inference.)
- T. Kumar, T. Dao, A. May. *Speculative Speculative Decoding*. ICLR 2026. (Parallelizes speculation and verification themselves, the technique’s own drafting step made speculative.)
- A. Samarin, S. Krutikov, A. Shevtsov, S. Skvortsov, F. Fisin, A. Golubev. *LK Losses: Direct Acceptance Rate Optimization for Speculative Decoding*. ICML 2026. (Trains draft models to directly maximize α instead of using KL divergence as a proxy; reports 8–10% gains in average acceptance length across models from 8B to 685B parameters.)

Together these cover the algorithm’s origin (2023) through 2026, with the two most recent entries showing the field is now optimizing the exact quantity, acceptance rate, this lesson centers on.

5. Teaching materials summary. The submitted ZIP contains three original artifacts. (i) A 16-slide lecture deck (`slides/spec_decoding_slides.pdf`) that motivates the memory-bandwidth bottleneck, walks through the draft/verify/accept-reject/resample loop phase by phase with a fully worked numerical example, reports measured speedups from the frontier literature (Leviathan et al., EAGLE, Medusa), addresses common misconceptions (the technique is not lossy, acceptance is probabilistic rather than a score match, and deeper speculation has diminishing returns), and closes with the Bloom’s-taxonomy learning objectives above. (ii) A self-contained JAX notebook (`speculative_decoding_lab.ipynb`) in two parts: a from-scratch implementation against synthetic categorical draft/target distributions that empirically verifies losslessness over 200,000 trials (runs on CPU in seconds, no downloads required), followed by a real-model benchmark on Hugging Face draft/target checkpoints (`distilgpt2 / gpt2-medium`) that measures wall-clock speedup on TPU hardware. The notebook is shipped pre-executed with all outputs visible for the CPU-only sections, and includes five exercises spanning remember through create: computing an acceptance probability and explaining the residual correction by hand, acceptance rate versus draft/target alignment, JIT-compiling the reference loop, a tree-structured (Medusa/EAGLE-2-style) extension, and a worked example of where the algorithm’s speed advantage breaks down while its correctness guarantee does not. (iii) A README with a suggested 75-minute lesson plan (lecture / hands-on lab / discussion) and instructor answer-key notes for all five exercises. All three artifacts were written specifically for this submission.